

Identifying Challenges and a Research Agenda for Flow in Software Project Management

Project Management Journal
Vol. 49(6) 1–16
© 2018 Project Management Institute, Inc.
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/8756972818800559
journals.sagepub.com/home/pmj


Denis Dennehy¹ and Kieran Conboy¹

Abstract

Flow and its associated tools and metrics are increasingly being reported as an approach used to achieve continuous deployment of software and delivery of value in software development projects. Yet, the extant literature on flow tools to date is rather limited. Also, what does exist tends to largely focus on the generic version of flow, rather than any rigorous examination of its use in the turbulent and constantly changing 'real world' environment within which it is applied. This study draws on a longitudinal case to identify the challenges of implementing flow and to provide a research agenda for flow project management.

Keywords

flow, Kanban, project management, software development

Introduction

Agile approaches continue to increase in popularity and have become the dominant mode of software development today (Ambler, 2014). Yet, problems such as poor preparation, poor project management, and insufficient understanding continue to exist in software projects, which has raised concerns of a second software crisis known as Software Crisis 2.0 (Hinchey, Fitzgerald, Donnellan, & Margaria, 2016). In response to this crisis, the concept of flow,* a core principle of contemporary lean thinking, is proving to be a catalyst for continuous improvement in software projects by providing an end-to-end focus that is unique to the lean methodology (Anderson, 2013; Anderson, Concas, Lunesu, Marchesi, & Zhang, 2011; Petersen, 2011; Reinertsen, 2009).

Rooted in lean manufacturing, flow approaches to software project management have emerged in recent years. There is strong evidence to suggest that awareness and use of flow approaches is gaining popularity across the project management community (Anderson, 2013; Dennehy & Conboy, 2017; Nord, Ozkaya, & Sangwan, 2012; Petersen & Wohlin, 2011; Poppendieck & Cusumano, 2012; Reinertsen, 2009). Flow is about managing a continuous and smooth flow of value-creating activities throughout the entire development process (Anderson, 2010; Petersen & Wohlin, 2011; Poppendieck, 2011; Reinertsen, 2009). Flow encourages (1) visualization of work as it passes through different states in the development process, for example, "Planned," "In Progress," and "Done"; (2) collaboration between teams; (3) measurement of value, costs, and technical metrics; and (4) knowledge sharing (Anderson, 2010; Bang, Chung, Choh, & Dupius, 2013;

Power & Conboy, 2015). Flow is useful for managing software projects because it emphasizes the continuous movement of valuable work, rather than a sequence of discrete activities, whether it is performed by distinct teams or in departments (Fitzgerald & Stol, 2015).

Flow differs from traditional software project management because it focuses on managing queues rather than project timelines and phases (Anderson, 2013; Poppendieck, 2011; Power & Conboy, 2015; Reinertsen, 2009). Flow differentiates itself from agile management approaches by (1) adopting an end-to-end perspective of the whole process (Anderson, 2013; Anderson et al., 2011; Petersen, 2011; Reinertsen, 2009), (2) moving from time boxed sprints to continuous deployment of software, and (3) limiting work in progress (WIP) (Cawley, Wang, & Richardson, 2013; Petersen, 2011; Sjøberg, Johnsen, & Solberg, 2012). Flow aims to maximize a continuous flow of value by (1) implementing WIP limits; (2) permitting new work items to immediately enter the development process, and, when required, prioritize it within the WIP limits; and (3) removing impediments to flow (i.e., time boxed sprints, context switching). It is also reported that flow techniques demonstrate higher productivity in comparison to time boxed

¹ The Irish Software Research Centre at National University of Ireland Galway, Galway, Ireland

Corresponding Author:

Denis Dennehy, The Irish Software Research Centre at National University of Ireland Galway, Galway, Ireland.
Email: denis.dennehy@nuigalway.ie

sprints in the Scrum method, which interrupt the fluent flow of work (Sjöberg et al., 2012).

Flow tools and metrics have been studied from various perspectives. Assimilation studies on Kanban (e.g., Anderson et al., 2011; Cocco, Mannaro, Concas, & Marchesi, 2011; Concas, Lunesu, Marchesi, & Zhang, 2013) have been conducted to test its application and effectiveness in software development. Results show that the application of flow tools such as Kanban and WIP limits have (1) improved control and management of workflow; (2) resulted in a more continuous, smooth flow of value; and (3) reduced the average lead time needed to complete customer requests. Other relevant studies have focused on Kanban adoption and usage (e.g., Anderson & Roock, 2011; Cocco et al., 2011; Concas et al., 2013), the effect of bottleneck detection (Senapathi, Middleton, & Evans, 2011; Shinkle, 2009), and visualization of development on metrics (Petersen, Roos, Nyström, & Runeson, 2014; Rutherford, Shannon, Judson, & Kidd, 2010).

Although flow research has increased in recent years, the existing body of knowledge is limited. First, most studies only focus on specific elements of flow (i.e., Kanban adoption, impediments to flow). Yet, it is well accepted that a software method such as flow cannot be studied in isolation (Ikonen, Pirinen, Fagerholm, Kettunen, & Abrahamsson, 2011). Second, a growing number of studies focus only on the functionality of a specific flow tool, such as Kanban, by using assimilation techniques, thereby excluding the social and contextual nature of software development projects. However, it is critical that flow tools are interpreted in the context of their use, as a failure to understand the context will limit understanding of how people use flow tools and the results (Kitchenham et al., 2010; Lyytinen & Rose, 2006; Olerup, 1991; Wastell & Newman, 1993). Third, the reported benefits of flow are largely based on anecdotal evidence that is driven by consultants who may be biased in favor of these results (Dybå & Dingsøyr, 2008; Ebert, Abrahamsson, & Oza, 2012; Sjöberg et al., 2012). However, it is still worth studying the use and potential effectiveness of flow approaches further, given that the rate of project failure across software development generally remains stubbornly high (Conboy, 2009; Ebert et al., 2012; Fitzgerald, Russo, & Stolterman, 2002; Kitchenham et al., 2002; Lyytinen & Rose, 2006; Petersen & Wohlin, 2009). This study addresses this gap in knowledge by pivoting between the intersection of practice and normative to empirically examine the challenges of implementing flow tools in a natural context, where they are intended to be used.

This longitudinal study will address the following questions:

1. What work practices are observed in the implementation of a flow-based software development process?
2. What challenges arise in applying flow tools for the management of information systems development (ISD) projects?

The article is structured as follows. First, we present the background of the concept of flow and the associated flow tools.

The research method and data collection techniques are then discussed. Next, the results and analysis of the research are presented, followed by discussion and a roadmap for future research. The article ends with conclusions and limitations of the study.

Background to the Concept of Flow

With origins in car manufacturing, the concept of lean traditionally focused on cost reduction (Hinchey et al., 2016; Standish Group, 2015), “the elimination of waste” (Ohno, 1988), and “doing more with less” (Naylor, Naim, & Berry, 1999; Ohno, 1988; Womack, Jones, & Roos, 1990). However, the concept of lean has evolved over time with subsequent emphasis on the value and, more recently, the flow of work. Lean strives to deliver maximum value to the customer by (1) reducing waste, (2) controlling variability, (3) maximizing the flow of information, and (4) focusing on the whole process rather than local improvements (Towill & Christopher, 2002). Lean is a mindset, a mental model of how the world works (Anderson et al., 2011; Poppendieck, 2011), and is guided by five interlinked concepts (Poppendieck & Poppendieck, 2013):

1. *Value*: Value in flow is defined by the customer and is used to determine what work is done and how it is prioritized.
2. *Value stream*: A value stream is a series of events that take a product or service from its conception through to the customer.
3. *Flow*: Flow refers to the continuous, smooth delivery of work through the development process.
4. *Pull*: Rather than developers pushing work through the system (e.g., to fill a two-week sprint), developers “pull” stories based on actual demand from the customer and the actual capacity of the team (e.g., when a developer finishes a story, he or she will “pull” in the next story).
5. *Perfection*: In flow, the team continuously reflects on the value stream, value delivered, and impediments, and continuously strives to improve until perfection is reached (i.e., perfect value is created with no waste).

Table 1 demonstrates how flow is a shared element of the seminal work on lean concepts in manufacturing as proposed by Womack and Jones (2010) and three variations of the lean methodology within the context of software development projects (e.g., Anderson, 2010; Poppendieck & Poppendieck, 2013; Reinertsen, 2009).

Reinertsen (2009) advocates that managing queue size, batch size, cadence, capacity utilization, and rapid feedback enable management to make economic decisions in managing software development projects. Unlike in manufacturing, where the lean methodology is used to “eliminate variation,” flow-based product development emphasizes the need for organizations to “exploit variation” because a side effect of trying to eliminate all variability is the elimination of innovation

Table 1. Principles of Lean Software Development Manifested in Different Approaches

Lean Concept (Womack & Jones, 2010)	Lean Software Development (Womack & Jones, 2010)	The Principles of Product Development Flow (Reinertsen, 2009)	Kanban Principles (Anderson, 2010)
Value	• Focus on customers	• Use fast feedback loops	
Value stream	• Eliminate waste • Optimize the whole	• Apply WIP constraints	• Limit WIP • Visualize workflow
Flow	• Increase flow	• Use cadence, synchronization, and flow control	• Measure and optimize flow
Pull	• Reduce friction	• Understand behavior of queues • Decentralize control	• Make process policies explicit
Perfection	• Keep getting better • Enhance learning • Energize workers • Build in quality	• Exploit variability • Reduce batch size • Use economically based decision making	• Use models to recognize improvement opportunities

Table 2. Impediments to Flow

Impediment	Description	Impact on Flow
Unnecessary motion	Is any movement of people, work or knowledge that is avoidable, that impedes the smooth flow of work, or that creates additional inefficiencies	Consumes resources that could otherwise be spent on value-adding activities
Extra processes	Generate extra work that requires extra time and effort	Consumes time and effort that could otherwise be spent on value-adding activities
Extra features	Features that are added without a verified need or valid hypothesis	Consumes resources that could otherwise be spent on value-adding activities
Delays	Situations in which something occurs later than it should	Add to the overall lead time from initial idea to delivery of product or service
Handovers	These occur when incomplete work must be handed over from one person or team to another	Impede flow by adding delays, requiring more people, and causing loss of knowledge as work is handed over to another team or person
Failure demand	Refers to demand placed on systems, teams, or organizations due to a failure to do something or to do something right for the customer	Leads to poor-quality work, which in turn results in the capacity of resources being used for non-value-adding work
(Too much) WIP	Work that is not yet complete, and therefore does not add value to the business or customer	Slows down the flow of work and delays the point at which value can be realized
Context switching	Occurs when individuals or teams divide their attention between more than one activity at a time	Adds lead time from initial idea to delivery of product or service, and causes failure demand
Unmet human potential	Refers to the waste of not using or nurturing people's skill set and abilities to their full potential	Flow of work and associated value created is neither effective nor efficient as it could be

within the organization (Reinertsen, 2009). This approach highlights a paradigm shift of product development that is enabling traditional organizational structures to give way to new team-oriented projects that are centered on the flow of value, not on functional expertise.

Flow is present in all seminal literature on the lean methodology and is the focus of this study. *Flow* is defined as “the progressive achievements of tasks along the value stream so that a product proceeds from design to launch, order to delivery, and raw materials into the hands of the customer with no stoppages, scrap, or backflows” (Womack & Jones, 2010, p. 306). Flow is a relatively new software development method that is increasingly being adopted by software teams (Poppendieck & Poppendieck, 2013). Managing a continuous and smooth flow is a key aspect of lean principles (Anderson et al., 2011). Adopting these principles results in software

development projects being based on forecasts, as commitment is delayed until demand is present to indicate what the customer really wants (Anderson, 2010; Petersen & Wohlin, 2011; Reinertsen, 2009).

Impediments to Flow

Impediments to flow are “anything that obstructs the smooth flow of work through the system and/or interferes with the system achieving its goals” (Power & Conboy, 2015, p. 2). Power (2014) identified nine types of impediments that impact flow in software development projects (see Table 2).

Identifying impediments to flow and assessing the quality of flow in software development projects is achieved using a number of flow tools and metrics. These are discussed in the next section.

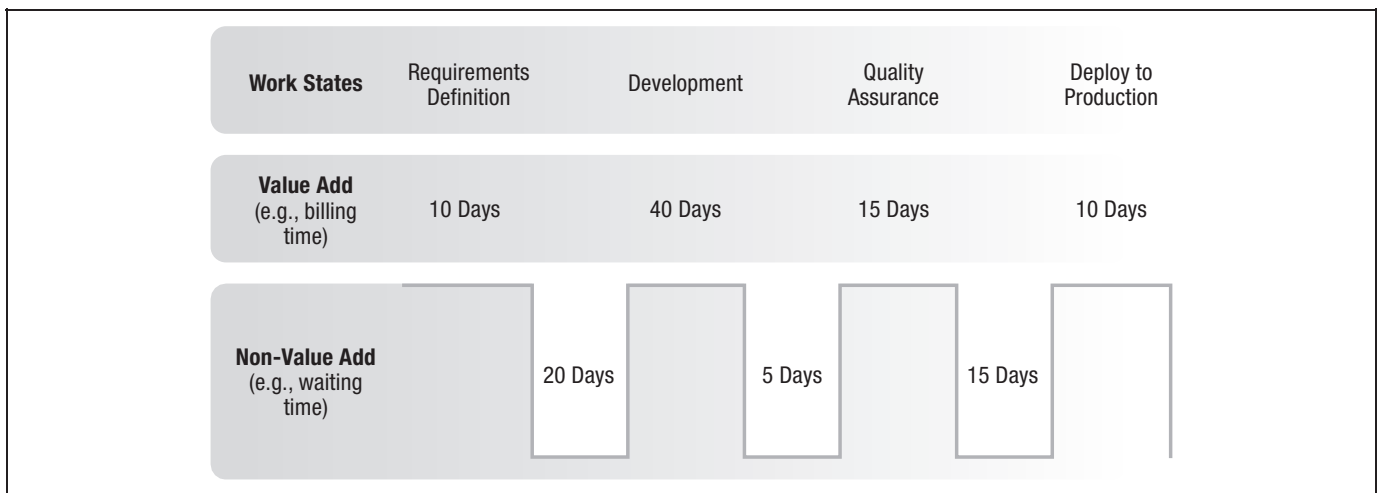


Figure 1. Value stream map.

Managing Flow in Software Development Projects

Understanding and managing flow in projects is achieved by using five commonly known tools, namely: (1) value stream maps, (2) Kanban boards, (3) cumulative flow diagrams (CFDs), (4) burn-down charts, and (5) line of balance status charts (Petersen et al., 2014).

Value Stream Map

A value stream map (see Figure 1) is used to follow a specified item of work through the process in order to establish the value added in each processing step (Petersen et al., 2014). Figure 1 is a value stream map that illustrates four work states (requirements definition, development, quality assurance, and deploy to production), the number of days (e.g., cycle time—the time the client is billed for the work) that a project team works on each work items within each state, and the number of days work items are idle between each work state (e.g., lead time—the time the client is not billed). Value stream mapping has been successfully applied in software development to reduce lead time (Petersen et al., 2014) and is useful for identifying value from four key perspectives—financial, customer, internal business process, and innovation (Mujtaba, Feldt, & Petersen, 2011). Although value stream mapping is useful for identifying impediments to flow retrospectively, it is limited in that it cannot be used to predict or prepare for impediments (Khurum, Petersen, & Gorschek, 2014).

Kanban Board

The Kanban board (see Figure 2) is generally project focused and encourages the visualization of workflow states as work moves through the organization by using a card system (Petersen et al., 2014). These cards enable team members to observe WIP, self-organize, and move work from a backlog through to done (Anderson, 2010; Petersen et al., 2014; Power & Conboy, 2015).

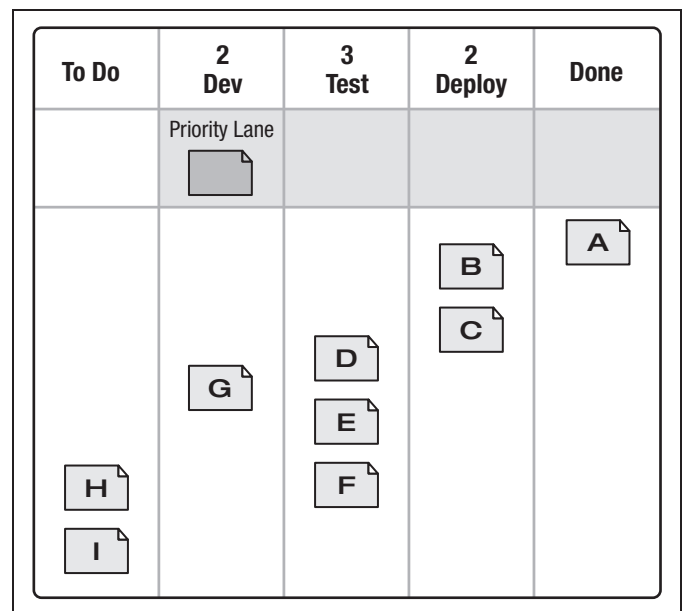


Figure 2. Kanban board.

2015). Each workflow state (i.e., To Do, Development, Testing, Deployed, and Done) contains a number of work items (story points, features) that then form a queue (Anderson, 2013; Anderson et al., 2011; Ikonen et al., 2011; Power & Conboy, 2015). For example, there may be a queue of 10 stories waiting to be developed but no stories in the testing queue.

Explicit WIP limits are used to manage the quantity of work at any given stage. Figure 2 shows a WIP limit of two story points (work items) in a development state, three WIP limits in a test state, and two WIP limits in a deploy state. A priority lane is established to allow a high-priority work item to be added to the workflow, if it does not exceed the WIP limit, or else another work item needs to be put on hold. Explicit policies (frequently called “entry” and “exit” criteria) are used to

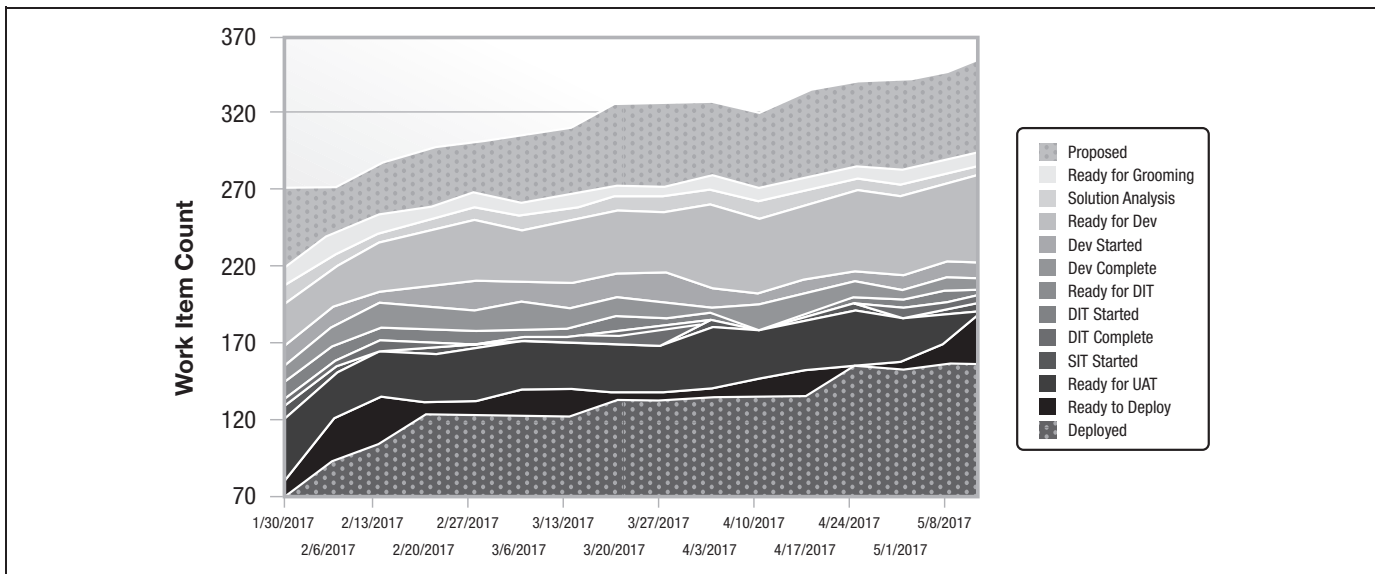


Figure 3. Cumulative flow diagram.

determine when a work item can be pulled from one state to another (Anderson, 2010). Kanban by itself does not guarantee success, as it is a relatively basic tool that needs to be supported by additional activities (Anderson et al., 2011; Polk, 2011).

Cumulative Flow Diagram (CFD)

A CFD (see Figure 3) provides an aggregated view of work in each of the defined work states. Rooted in queuing theory, the CFD is useful for understanding the behavior of queues and for diagnosing problems (Ikonen et al., 2011). A limitation of the CFD is that it is backward rather than forward looking (Power, 2014; Reinertsen, 2009).

Figure 3 illustrates 11 work states through which all story points (work items) progress from the initial “proposal” state to “deployed:” (1) ready for grooming, (2) solution analysis, (3) ready for development, (4) development started, (5) development complete, (6) ready for development integration testing (DIT), (7) DIT complete, (8) system integration started, (9) ready for user acceptance testing, (10) ready to deploy, and (11) deploy.

Burn-down Chart

A burn-down chart (see Figure 4) is used to show an aggregated view of the amount of work that remains versus what was initially estimated (Petersen et al., 2014). It is useful for predicting when will work will be complete, given the current pace of the team, and so it is often used to schedule or move future meetings or demonstrations with customers, and to predict if evasive action is needed to avoid releases over key periods, such as holiday periods or accounting year-end. A limitation of a burn-down chart is that it is difficult to identify if a specific work item is soon to become a flow impediment (Kniberg & Skarin, 2010; Petersen et al., 2014).

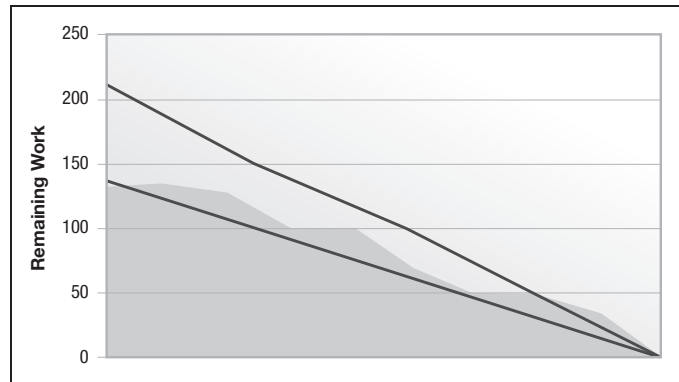


Figure 4. Burn-down chart.

Line of Balance Status Chart

The line of balance status chart (see Figure 5) is a tool that allows the balancing of operations such that each activity is continuously and effectively performed (Petersen et al., 2014). The line of balance status chart is backward looking and presents an aggregated view (Arditi, Tokdemir, & Suh, 2001, 2002).

Complementary to these flow tools are four key metrics used for understanding and managing flow in projects (Petersen et al., 2014): (1) *Queue size* identifies the number of pieces or work remaining, which can cause of various forms of economic waste, such as longer cycle time, increase risk, increase overhead, and lower quality; (2) *cycle time* shows how long individual work items spend in specific workflow states and is used to understand how work flows through individual or combinations of work states; (3) *lead time* shows how long it takes for individual work items to move through a system, from initial request to delivery to end user; and (4) *throughput rate* reveals the rate of work through the system over time.

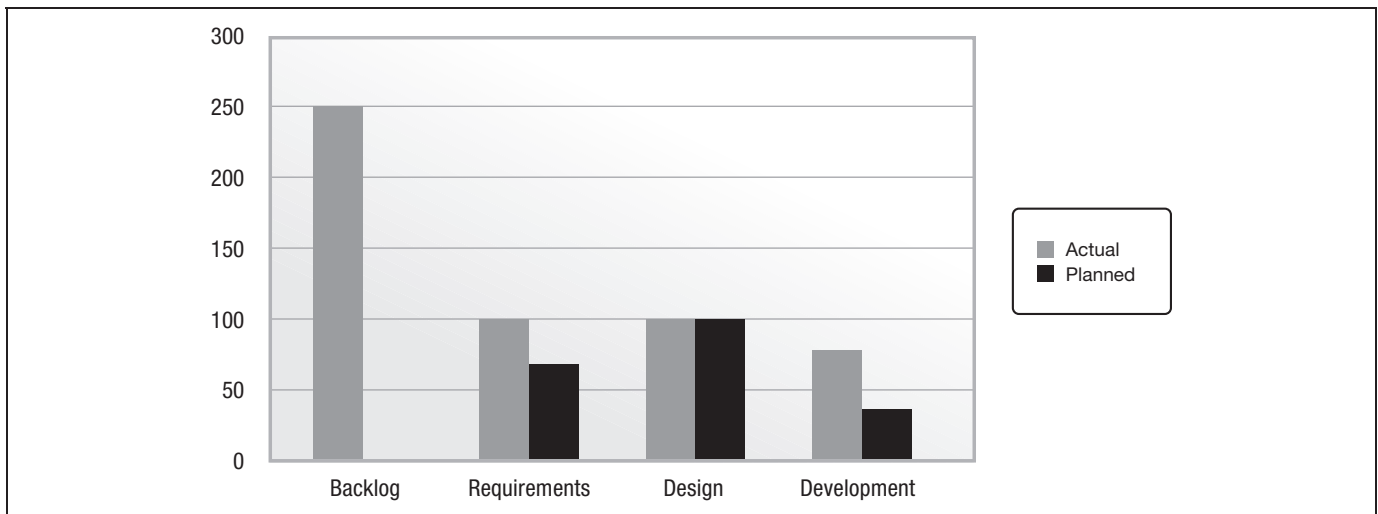


Figure 5. Line of balance status chart.

Reinertsen (2009) posits that because flow is about identifying different causes for success, it follows that flow emphasizes cycle-time and lead-time metrics. Flow approaches do not assume a set of homogeneous projects or similar types of work. In a similar manner to agile approaches, flow deconstructs work of different types into sets of user stories and then into similar-sized pieces of work. Small stories may be represented by one task, whereas major, complex, “epic” stories may be broken into hundreds of smaller and more consistently sized tasks.

Research Method

This study uses an exploratory, single case study method involving four software development teams. A case study was chosen because it is suited to “practiced-based problems where the experiences of the actors are important and the context of action is critical” (Benbasat, Goldstein, & Mead, 1987, p. 369). Adopting this method is valuable for understanding a new phenomenon that lacks empirical substantiation (Yin, 2009), identifying how certain conditions change over time (Kniberg & Skarin, 2010), and engaging with those who are “living the case” (Eisenhardt, 1989). This case was purposefully selected for two reasons pertinent to this study. First, the four software development teams being studied were adopting the flow approach in a distributed environment with continuously changing business requirements. Second, there was continued direct access to the participants involved in the flow projects selected, which included ongoing informal interviews, observations, and access to relevant project documentation. This proved highly valuable when studying flow, which, as stated earlier, is highly context-dependent and socially embedded.

Case Background

The organization studied is an industry leader in technology solutions services and support; its global workforce exceeds

140,000 people, of which 23,000 are located at the Irish campus, and 5,000 of these staff are involved in the development of IT systems and software.

The four teams studied are distributed, with members based in Ireland, India, North America, and Brazil. The teams have been using the Scrum method and Microsoft Team Foundation Server (TFS), a commonly used code management tracking software tool for five years prior to the adoption of the flow approach. The flow tools and metrics that were used include value stream maps, physical and virtual Kanban boards, CFDs, burn-down charts, cycle time, and lead time. The differences between flow and Scrum are listed in Appendix A.

The initial decision to examine the potential of flow approaches was taken 12 months prior to this case study being conducted. Certain senior staff recognized a need for a continuous, smooth flow of work, which was not being realized with the current software methodology. There was a lack of visibility of the work being conducted, as well as a perceived ad hoc approach to the estimation, development, and release of software generally. There was a belief that flow could (1) provide greater visibility into what the development teams were doing, (2) promote performance improvement, (3) improve feedback loops, and (4) expose resourcing constraints and capacity utilization.

Data Collection and Validity

In this study, two researchers collected data between October 2015 and January 2017 (15 months). Interviews, direct observations, participant observation, and physical artifacts (i.e., Kanban boards) were used as sources of evidence. These techniques are well suited to case study research, and particularly for exploratory research, because they allow expansive discussion to illuminate factors of importance (Yin, 2009). They also increase immersion within the broader context of the case (Stake, 2000). Advantages of using multiple sources of

Table 3. Interviewee Profile

No.	Interviewee Job Title	Team	Years of ISD Experience
1	IT director	Management	25
2	Portfolio manager	Management	30
3	Financial controller	Management	21
4	Systems integration manager	Management	20
5	Software development consultant	Management	18
6	IT delivery manager	Management	16
7	Senior business analyst	Management	12
8	Project manager	Team 1	15
9	Software developer	Team 1	12
10	Software developer	Team 1	2
11	Project manager	Team 2	15
12	Software developer	Team 2	2
13	Software tester	Team 2	3
14	Project manager	Team 3	21
15	Software tester	Team 3	2
16	Project manager	Team 4	12
17	Software developer	Team 4	15
18	Software developer	Team 4	2

evidence include the increased likelihood of acquiring a rich data set, capturing the contextual complexity of the case, and developing “converging lines of inquiry”—a process of triangulation that increases the precision of the empirical research, thus making the findings and conclusions more convincing (Oppenheim, 1992; Yin, 2009).

Eighteen interviewees were selected using role-based sampling (Glaser & Strauss, 1967; Runeson & Höst, 2009). Interviewees varied in terms of role, industry experience, and knowledge of information systems development (ISD). The interviewees’ characteristics are listed in Table 3.

An interview protocol was prepared based on the flow approach, and specifically the subcomponents of flow: value, value stream, flow, pull, and perfection (see Table 1). These provided a list of “intellectual bins” (Benbasat et al., 1987; Denzin & Lincoln, 2011; Remenyi & Williams, 1995; Runeson & Höst, 2009; Yin, 2009) to structure the data collection and for thematic analysis. A sample of the data analysis is shown in Appendix B. Questions were largely open-ended, allowing respondents the freedom to convey their experiences and views, and to express the socially complex contexts that underpin their organizational context.

Interviews were conducted in a reflexive manner (Ikonen, Kettunen, Oza, & Abrahamsson, 2010), allowing the researchers to adjust the content and schedule within and between interviews. Interviews lasted between 40 and 90 min, and were recorded, transcribed, proofread, and annotated.

To achieve *internal validity*, both researchers took observation notes to aid triangulation and discussion with the practitioners, and the interviews were supplemented by more than 120 hr of direct observations of the four software teams. This enabled the researchers to identify inconsistencies between the information reported and actual actions (Miles & Huberman,

1984). In terms of *conclusion validity*, conclusions drawn in this study were based on the best assessment of the case data that were gathered and analyzed. The results of the data collection were discussed with the practitioners interviewed and with professional colleagues. Future research will test this further to see if differing perspectives shed additional or even contradictory light on the findings. *External validity* or “generalizability” (Maxwell, 1992) is challenging for qualitative research, particularly for case studies such as this that embrace the complexities of context (Yin, 2009). Therefore, we did not seek external validity. However, we did collect and present as much of the contextual data as feasible to enable readers to judge whether the conclusions of this study may be relevant in other software development contexts.

Results and Analysis

This section presents the reported benefits of flow by the organization studied and the challenges that emerged in applying flow tools for the management of ISD projects.

Following an internal review of the flow implementation initiative, a project manager reported, “After being exposed to flow tools such as Kanban and CFDs, we started focusing the teams’ mindset toward to a more flow-based approach. We have rebuilt the entire software development life cycle tool with queries and dashboards, which is helping to visualize work in progress, and help the team focus on priorities rather than just delivering work. This weekend, we delivered 58 work items produced in four sprints, which is almost three times our productivity before adopting flow.”

Another project manager reported, “The visualization of work has really improved on the new flow-based customizations performed in the actual virtual flow tool.” A similar view was shared by an experienced software tester who reported having a better understanding of work practices that are in place for his team: “Visualization was the biggest improvement, especially with Kanban, as it allowed us to see priorities and blockers quickly so that we could react and make decisions faster.”

A potential benefit of flow is the facilitation of efficient hand-offs as shared by this software developer: “What we can expect is that flow can help us to streamline the activities to avoid panic mode, grow confidentiality, and bring more clarity with hand-offs, which is vital because of the way our current agile process is structured.”

The benefits of implementing flow had a positive impact beyond the project teams. A manager responsible for regional operational excellence stated: “This is really great—it started out from being a pilot project to broadening out to cross-functional teams, with cross-regional interest and collaboration with our customers.”

Table 4 provides evidence of both tangible and non-tangible benefits that were reported across the four software project teams. The quantitative benefits were extracted from TFS, which consisted of stories (e.g., work items) that were

Table 4. Benefits of Flow-Based Software Project Management

Stories and Defects	Visibility and Quality	Planning and Allocation
• 100% of stories compliant with TFS work states • Average lead time reduced from 109 to 39 days • Average cycle time reduced from 31 to 24 days • One in eight stories had a defect (previously one in five stories had a defect)	• Greater ownership of defects • Defects now visible during daily standup • Quality control checks integrated throughout each sprint • Greater visibility of defects not being closed promptly	• Team proactively resolving defects rather than waiting to be assigned • Improved planning and resource allocation to ensure defects are closed promptly • Improve communication between development and testing teams

Table 5. Application of Lean Concepts Across the Software Teams

Concept	Team 1	Team 2	Team 3	Team 4
Value	Value was clearly defined by the end consumer	Value was clearly defined by the end consumer	Value was clearly defined by the end consumer	Value was clearly defined by the end consumer
Value Stream	Value stream maps created to understand process and identify flow impediments	Value stream maps created to understand process and identify flow impediments	No value stream map was used by this team	No value stream map was used by this team
Flow	Flow approach was consistently used	Flow maintained with exception of interrupts from two-week sprints	Flow maintained with exception of interrupts from two-week sprints	Flow approach was consistently used
Pull	Pull approach slowly replaced push approach	Pull approach slowly replaced push approach	Pull approach slowly replaced push approach	Pull approach slowly replaced push approach
Perfection	Evidence that team was committed to double-loop learning	Evidence that team was committed to double-loop learning	Evidence that team was committed to double-loop learning	Evidence that team was committed to double-loop learning

completed by the project teams before and after the adoption of flow tools. The qualitative data was collected via interviews with management and software teams.

Although these empirical findings offer promise for the use of flow tools and metrics, some of the project teams encountered managerial and technical challenges during the implementation of flow. These are presented in the next section.

Managerial and Technical Challenges

As highlighted previously, the lean methodology is guided by five interlinked concepts: value, value stream, flow, pull, perfection (Womack & Jones, 2010). The seven key challenges that emerged from the case can be considered symptoms of these concepts not being fully enacted within the case (see Table 5). Further evidence to support this analysis is presented in Appendix B.

This section presents seven key challenges that emerged from the case studied: (1) integrating flow with existing processes, (2) over-control versus the need to monitor, (3) changing to a flow-oriented organizational culture, (4) establishing and maintaining a Kanban board, (5) establishing WIP limits, (6) calculating and interpreting flow data in context, and (7) tailored flow training.

Challenge #1: Integrating flow with existing processes. Because the organization had adopted Scrum for the previous five years, there was resistance to replacing it with an alternative. For this reason, teams attempted to blend flow techniques with Scrum. However, blending flow (which promotes continuous development) with Scrum (which uses time-boxed sprints) was a major challenge across teams. For example, a software developer commented, “Since we are trying to facilitate a hybrid between Kanban and Scrum, we find that flow on the Kanban board often stalls due to the focus on sprint work.” A project manager also noted: “It takes time to fully embed the changes into the team, and asking them to change and align with a flow method will be a challenge.”

Bureaucracy and technological constraints hindered changes to the development approach. A software developer said that “all aspects of process changes go through a change management board before anything is really changed.” Further evidence of this challenge was experienced when another team attempted to create a virtual Kanban board within the code management software, TFS. A project manager explained, “We only want to track defects on a Kanban board, but TFS did not allow for us to do this efficiently, so we ended up building our own solution.”

Challenge #2: Over-control versus the need to monitor. Flow is metric oriented and emphasizes transparency and visibility

of work to all, including management. Some interpreted this as a management control mechanism and subsequently resisted. A project manager noted that because the organization already had other “control metrics in the background that were used systematically to monitor, for example, the weekly percentage of team utilization . . . and then flow metrics would be perceived as another control.” Monitoring is required in a flow context, as one manager explained: “Flow provides greater visibility into what teams are doing . . . it will definitely improve the feedback loops and it will expose resourcing constraints, as well as capacity utilization.” His colleague shared a similar view that flow “exposes people, but not expose them in a bad way to say that person is not doing their job.” Another manager also believed in the value of monitoring and suggested that “teams need to see from a management standpoint and from a team standpoint that progress is being made and they can measure how well they are doing, or not, and how we improve.”

Challenge #3: Changing to a flow-oriented organizational culture. As highlighted previously, flow approaches require a change in mindset among managers. Enacting flow requires project managers to reframe how they manage project teams, which can be challenging when the dominant culture was to utilize resources to full capacity. For example, a manager explained that the previous organizational culture was “to load a team up, if they have 20% spare capacity.” This challenge was echoed by his colleague, who explained that “no people would be left idly waiting to solve problems if they worked 100% of their time on a billable customer project.” Changing to a flow approach takes time, as highlighted by a software tester: “When a release is looming, we drop flow—we shift focus to the release. We need a connection between the latest activities and the flow tools. We are missing that connection.” His fellow team member commented, “When something like having a high-priority release at short notice occurs, flow goes out the window,” also highlighting this cultural challenge.

A historical culture of fear and fear of change exacerbated this challenge. As one manager commented, “If implementing flow results in savings of X dollars, people are going to wonder if they will be out of a job.” A suggestion on how to change the organizational culture was provided by another manager: “The real way to affect culture and introduce change is to affect behavior, which will then affect culture. However, in a large multinational, the only way to affect culture that I’ve seen is to demonstrate metrics with dollar savings.”

Challenge #4: Establishing and maintaining a Kanban board. One of the first steps to managing flow is to establish a physical or virtual Kanban board where the work is visually tracked and managed. This is challenging, as it requires input and consensus from people working across the team, project, or program of work in question. A manager explained, “We had to sit with all members of each group from each program to try and structure this. We needed to define these states and agree that this is

what we all follow.” As the teams initially started with physical Kanban boards, some team members believed it created extra work, as reported by a project manager: “There is a huge amount of work in the manual maintaining of Kanban. Ideally, you should be able to do it all electronically and visualize it on a large digital screen.”

Once established, maintaining a Kanban board is still challenging. A project manager from team 2 explained that team members are “very focused on what they have to do in the current sprint and are not looking at the Kanban board.” Although setting up and maintaining a Kanban board is challenging, it does provide benefits to the team. During a review of the project, a project manager reported, “Kanban is helping the team to focus on priorities rather than just delivery work. This weekend we just delivered 58 work items produced in four sprints, which is almost three times our productivity before we adopted flow.”

Challenge #5: Establishing WIP limits. Once work is represented on the Kanban board, it is necessary to manage the amount of WIP, and this is achieved by establishing WIP limits. Because there is no universal formula for establishing WIP limits, the process requires trial and error, which can be challenging for management and project teams. A manager explained that the team must “figure out what is our optimum WIP limit, how does that reconcile with geographical boundaries, and the competence domain because sometimes these competencies are not automatically transferable.” A similar view was expressed by his colleague who stated, “We are not clear how to enforce it, and stop work moving over as there isn’t somebody standing at a gate. We are still figuring out how to do this rather than deploy it as a blunt instrument that becomes dysfunctional.” Because of the cultural challenge discussed previously, teams were hesitant to establish WIP limits, as they believed that management would not adhere to them because they continued with a “push demand,” rather than “pull demand” by the customer.

Challenge #6: Calculating and interpreting flow data in context. It is important that flow metrics are interpreted in context because failure to do so limits the understanding and the usefulness of the results. For example, one interviewee explained, “Metrics do not show the opportunity for innovation in solutions.” There was concern that management could misinterpret cost of delay (COD), a key metric used in flow. One manager said, “If I explain to my manager that one project has a higher COD than another, they would laugh at me because the decision was made at a senior level that impacts other things that it would all have to be done again as these projects are prioritized in advance. It would be like, where were you two years ago?” Adding to this challenge, the research team observed that two teams manually extracted data from TFS to calculate flow metrics. This contributed to frustration; as one project manager commented, “Metrics should be taken from TFS because the only way to

embed a process is to automate it. If it's not in TFS, it's going to fail."

Challenge #7: Tailored flow training. The ability to apply flow in software development projects requires more than a standard training program. It also requires people who can interpret and communicate flow metrics to different audiences, as flow metrics hold different meaning for management and developers. For example, one manager noted, "There is no one-size-fits-all training package." Another manager commented, "I can see training being an issue. It is vital that we get the right people because if you train the wrong people, it won't work." A project manager said, "People are still very dependent on the project manager to show them how to use flow tools to understand what work is done or prioritized."

Discussion and Research Agenda for Future Research

This section provides a research agenda used to overcome the challenges of flow in project management. The research agenda contains a set of research questions emerging from the reported challenges of flow adoption at the case studied. These are listed in Table 6; for each challenge, we set out a range of example research questions.

The earlier literature review covered the existing general research. These flow challenges, however, emerged from the data in this case study and so we now reflect on existing research in relation to each challenge to distinguish between a new contribution and what is already known.

Integrating flow with existing processes, specifically with Scrum (time boxed sprints) was challenging, as it interrupted the fluent flow of work. It has been reported that Kanban is a lightweight intuitive tool (Rubin & Rubin, 2005; Wengraf, 2001) that has been merged with Scrum in the form of Scrumban. However, flow is not only about visualizing work items on a board; it is also about setting up WIP limits and limiting the size of work items. Further, flow is more than just using Kanban; it is about the smooth and continuous flow of work, which is not achieved when integrating with the two-week sprint, which, we know from this study, interrupted the flow of work.

Over-control versus the need to monitor emerged as a legitimate challenge. Identifying the control modes present in flow was outside the scope of the research. Previous research (Kirsch, 1997; Robinson, Segal, & Sharp, 2007) examined the application of control modes within the context of software projects. However, this is the first study to identify control as a concern in flow tools and practices. Understanding what control modes are present in flow tools and how these modes are enacted in flow-based projects remains understudied. From a research perspective, future researchers may wish to apply control theory to study flow tools in software development projects, as this will inform both management and project teams on how to effectively balance formal and informal control modes (cf. Harris, Collins, & Hevner, 2009). From a

Table 6. Research Agenda for Flow Research

Flow Challenge	Example Research Questions
1. Integrating flow with existing processes	- How can flow and Scrum be combined effectively? - How are conflicting practices or metrics resolved? - How can a combination of flow and Scrum be scaled to (1) large teams, (2) distributed teams, and (3) portfolios of projects?
2. Over-control versus the need to monitor	- What control modes are present in flow tools? - How are control modes enacted in flow-based project? - How can flow controls be balanced to allow both flexibility and required control?
3. Changing to a flow-oriented organizational culture	- Who are the right people to lead organization-wide flow initiatives? - What lessons can be learned from the adoption of other ISD methods?
4. Establishing and maintaining a Kanban board	- How does the organization's use of time impact the use of Kanban boards? - How can Kanban boards be used at the portfolio project level?
5. Establishing WIP limits	- What rules need to be established to set up WIP limits? - Do WIP limits stabilize over a period of time?
6. Calculating and interpreting flow data in context	- How can flow data be communicated between project and management teams? - How can flow data be used for planning and process improvement? - How can contextual data be incorporated into flow processes and used to impact and adapt decision making?
7. Tailored flow training	- How can flow-training programs be tailored for project and portfolio level teams, and business units? - What challenges does this create for trainers?

practice perspective, organizations should ensure that flow is considered within its wider portfolio of control modes. Therefore, it is important to determine whether the method itself is suitable in that instance.

Changing to a flow-oriented organizational culture was influenced by the organizational history of the organization (i.e., job redundancies) and the mindset of managers (i.e., operating at full capacity). Although flow tools and metrics can be implemented quickly, using them as a mental model of how the world works inevitably will be incremental, as changes in

culture occur over a period of time (Law & Lárusdóttir, 2015; Mahnic, 2015; Middleton & Joyce, 2012). Previous studies (e.g., Choudhury & Sabherwal, 2003; Chua, Lim, Soh, & Sia, 2012; Conboy, 2010; Drummond, 2011; Kirsch, Sambamurthy, Ko, & Purvis, 2002; Drummond, 2011; Ply, Moore, Williams, & Thatcher, 2012; Wiener, Remus, Heumann, & Mähring, 2015) have indeed highlighted the importance of leadership, communication, and team learning to successfully use flow tools such as Kanban.

Establishing and maintaining a Kanban board was identified as a key challenge. This can be expected because the contexts associated with contemporary software project management (Birkeland, 2010) differ for each team. Therefore, the work states and definitions used for each Kanban also differs. Although previous studies rigorously examined Kanban adoption and usage (e.g., Ikonen, 2010; Leopold & Kaltenecker, 2015; Nikitina & Kajko-Mattsson, 2011; Polk, 2011; Stray, Moe, & Dingsøyer, 2011), they were not longitudinal studies that examined how teams move from adoption to infusion (cf. Gallivan, 2001). This is of particular importance when adoption decisions are made at the organization or departmental level rather than at the individual level. Other studies (e.g., Anderson et al., 2011; Cocco et al., 2011; Concas et al., 2013) examined Kanban in controlled experiments (i.e., simulation), but these do not reflect the complex nature of software project management.

Establishing WIP limits is a key challenge, as there is no prescribed approach to finding the optimum WIP limit. The organization not consistently adhering to the “pull demand” element of flow exacerbated this challenge. Although there is literature on establishing WIP limits (Fichman & Kemerer, 1997; Orlikowski, 1993), none has provided a solution for setting WIP limits in the constantly changing and complex environment in which software project teams operate. Although setting WIP limits is challenging, this study highlights that establishing and adhering to explicit entry and exit policies for each work state on a Kanban board is equally important.

Calculating and interpreting the flow data in context was challenging for a number of reasons (manual process, time-consuming). This may suggest that these metrics should first be embedded within existing systems with the support of people who create metrics on a daily basis. Current flow literature (e.g., Anderson et al., 2011; Cocco et al., 2011; Ikonen, 2010) emphasizes the importance of flow metrics in order to identify the quality of flow (e.g., Birkeland, 2010). However, this study highlights that the application of these metrics in a comprehensive and rigorous manner remains a problem for project teams, which may undermine the value add of flow in software projects. A related problem is that flow has not yet been successfully scaled from the project to the portfolio level; this could lead to flow tools and metrics being used in isolation by project teams, which could limit the potential benefits of the flow method. As highlighted in Table 6, future research is warranted to ensure that flow is scaled to the portfolio level.

Tailored flow training was identified as a key challenge for software and management teams, mainly because there was a tendency to oversimplify the flow concept, which can lead people to think that they fully understand flow and then make misinformed judgments about flow tools and metrics. Although tailored flow training is important, it is not like learning a software programming language, where, once learned, it can be easily applied universally. In contrast, the essence of flow is about taking a panoptic view of the flow in the software development process, in order to make economically informed decisions. This requires an understanding of context at both the project and portfolio levels.

Conclusions and Limitations

In spite of the widespread adoption of agile and Scrum-type software development methods, their shortcomings have led to a continued search for improved development methods. Flow is an emergent method that holds potential, but this will be realized only if some of the issues and difficulties are successfully addressed. Although flow is gaining rapid popularity and is supported by anecdotal claims of effectiveness, real-world cases using the flow approach are still relatively rare. This empirical study provides two important contributions to flow research and practice. First, it identified a set of empirically based challenges, drawing on a case where the flow approach had been implemented. We acknowledge that of the seven challenges identified, “organizational change” and “setting up Kanban boards” have been identified by previous studies (Klipp, 2011; Kniberg & Skarin, 2010; Reinertsen, 2009). However, aside from the new challenges identified, this is the first study to provide such an overarching set of challenges in a single study. Second, this study provides a research agenda for improving the understanding and effectiveness of flow in software project management.

There are a number of limitations to this research, which should be taken into consideration. First, it should be noted that although this was a 15-month longitudinal study, it is based on a single case with four software teams that were still gaining experience in the flow approach. Though this was a very useful setting for studying flow challenges, an advanced “expert” case could add valuable insights. Second, because the findings are based on single case, it is difficult to establish how many of the findings pertain to the flow method in general and how much they pertain to the idiosyncrasies of this case and its employees. Future research examining flow-based project management across a number of cases would address this limitation. Third, the researchers attempted to develop an exhaustive set of example questions to set a research agenda for flow project management. Selection criteria included novelty, urgency, and impact of questions. Other researchers and practitioners may consider adding or editing questions, or tailoring them to their own industry or project context.

Finally, although the flow approach is increasingly prevalent in systems development projects and evidence

suggests that the approach is effective in practice, such methods are only one mechanism with which to manage software projects. From a practice perspective, organizations should ensure that the flow method is considered within this

wider method portfolio. Therefore, before measuring the challenges and benefits of the flow method at a team level, it is important to determine whether the method itself is suitable in that instance.

Appendix A: Differences Between Flow and Scrum

Note: Though cognizant of the limitations of drawing comparative conclusions between methods, in this study, the authors have built on previous analysis by Kniberg and Skarin (2010) to list the differences between flow and Scrum methods.

Flow	Scrum	Literature Sources
Work is pulled through the system using single-piece flow	Work is pulled through the system in small batches	Power and Conboy (2015)
Cadence is a continuous flow (time boxed iterations optional)	Cadence is based on time boxed iterations (two- to four-week sprints)	Kniberg and Skarin (2010)
Work in progress limits are explicit	Work in progress limits are implicit	(Reinertsen, 2009; VersionOne, 2016)
Roles are required but are prescribed by the team rather than the method	Prescribed roles: product owner, scrum master, development team	(Kniberg & Skarin, 2010; VersionOne, 2016)
Kanban board is used to visualize work states and is in continuous use	Scrum board is optional and is reset at the end of a sprint	(Birkeland, 2010; Reinertsen, 2009)
Cross-functional team optional (specialist teams permitted)	Cross-functional teams prescribed	(Kniberg & Skarin, 2010; VersionOne, 2016)
Size of work items can vary as there is no explicit rule that it must be completed within specific time box	Size of work items are broken down in sizes that must fit a sprint (i.e., two-week sprint)	(Birkeland, 2010; Kniberg & Skarin, 2010)
Release methodology is at the team's own discretion	Release methodology is at the end of each sprint, if approved by product owner	(Kniberg & Skarin, 2010)
Lead time is the default metric for planning and process improvement	Velocity is default metric for planning and process improvement	(Birkeland, 2010)
No specific chart prescribed (CFD is generally default chart)	Burn-down chart is prescribed	(Cocco et al., 2011; VersionOne, 2016)

Appendix B: Sample of Coded Data

Value	Value Stream	Flow	Pull	Perfection
There is another level of visualization required and that's not there at the moment, we're only visualizing effectively the high-level requirements....That is the biggest difficulty, setting out the roles, responsibilities, and the process, the ways of working around that so people are actually aligned with producing stuff.	I personally believe that any metric that involves people having to enter data into a sheet is waste; if it's done correctly, metrics should be taken from application life cycle management tools. Metrics need to be collected and gathered within the tools that we use and if they're not, even if you got somebody to fill in a spreadsheet today, two weeks from now or they get a new manager saying, "We don't have time for that; leave it."	We have huge challenges. The flow, we need to have control of what comes in and what comes out. I know that and we need to do it, and also my teams are becoming available at different times and I need to understand it.	You have the conflict between your traditional project planning and your flow through the system. You can study all you like, but it is going into a clogged pipe—no point pushing more in and that would have been a mindset before in that, in the past, you would have a team working on requirement X but we would say requirement Y needs to be delivered in this release so we better start with Y, and it typically needs four months of development and there was a feeling that once we started it, now we have a chance of making it.	I think there is a fundamental belief that it will improve productivity but how ever in the name of God do you measure productivity in software anyway? I just don't believe it is possible. I just think organizations can put too much effort into measuring stuff, and if you put that energy into doing stuff. Look, we are all engineers; we like to see metrics and these tangible actions and tangible results, but we are also dealing in the creative economy.

(continued)

Appendix B (continued)

Value	Value Stream	Flow	Pull	Perfection
Our handovers take more time to just get feedback about what I deployed or coded today. The end user might get annoyed; we cannot deliver the value even if we fix it because it takes some release for it to be aligned.	I've always been a believer when it comes to process, including metrics, that the only way to truly make a process embedded in an organization is to automate it. If it's not in a tool, it's going to fail.	I believe flow will not just benefit teams, it will benefit the organization end to end; that's where it needs to be engaged with and brought through by everybody or everybody in a position of influencing. It's going to take time, I think, and that's the challenges we face on this. How we get around that and nurture their interest and get people into it more, that's a challenge in itself.	[In terms of WIP limits] they are experimenting with those because...we are not clear how to enforce it, stop stuff moving over because the teams are out there and they know they stuff is coming and they will start work. There isn't somebody standing at a gate, so we are still figuring out how to do that and deploy that rather than deploy it as a blunt instrument that becomes dysfunctional.	We've set up an operations review on a monthly basis using this data like the CFD and the control charts. The take-up has been poor; we have only set up one activity so far, but it's all got to do with understanding how to use it and in some cases because of the up-front planning that has taken place, we have set out a roadmap or a kind of up-front plan.

Declaration of Conflicting Interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) disclosed receipt of the following financial support for the research, authorship and/or publication of this article: This work was supported with the financial support of the Science Foundation Ireland grant 13/RC/2094 and co-funded under the European Regional Development Fund through the Southern & Eastern Regional Operational Programme to Lero—the Irish Software Research Centre (www.lero.ie).

Note

* The concept of flow that has emerged in the software development literature is distinct from the psychological concept of flow, which refers to a state of concentration or complete absorption within an activity (cf. Csikszentmihalyi, 1975, 1991).

References

- Ambler, S. (2014). *2014 agile adoption survey*. Retrieved from <http://www.amblysoft.com/surveys/agileJanuary2014.html>
- Anderson, D. (2010). *The Kanban principles*. Seattle, WA: Blue Hole Press.
- Anderson, D. (2013). *Lean software development*. Seattle, WA: Lean Kanban University (LKU).
- Anderson, D., Concas, G., Lunesu, M. I., Marchesi, M., & Zhang, H. (2011, May). Studying lean-kanban approach using software process simulation. In A. Sillitti, O. Hazzan, E. Bache, & X. Albaladejo (Eds.), *International conference on agile software development* (pp. 12–26). Berlin: Springer.
- Anderson, D., & Roock, A. (2011). An agile evolution: Why Kanban is catching on in Germany and around the world. *Cutter IT Journal*, 24(3), 6.
- Arditi, D., Tokdemir, O. B., & Suh, K. (2001). Effect of learning on line-of-balance scheduling. *International Journal of Project Management*, 19(5), 265–277.
- Arditi, D., Tokdemir, O. B., & Suh, K. (2002). Challenges in line-of-balance scheduling. *Journal of Construction Engineering and Management*, 128(6), 545–556.
- Bang, S. K., Chung, S., Choh, Y., & Dupius, M. (2013, October 10–12). *A grounded theory analysis of modern web applications: Knowledge, skills, and abilities for DevOps*. Proceedings of the 2nd annual Conference on Research in Information Technology, Orlando, FL. Retrieved from <https://dl.acm.org/citation.cfm?doid=2512209.2512229>
- Benbasat, I., Goldstein, D. K., & Mead, M. (1987). The case research strategy in studies of information systems. *MIS Quarterly*, 11(3), 369–386.
- Birkeland, J. O. (2010). From a timebox tangle to a more flexible flow. In A. Sillitti, A. Martin, X. Wang, & E. Whitworth (Eds.), *International conference on agile software development* (pp. 325–334). Berlin: Springer.
- Cawley, O., Wang, X., & Richardson, I. (2013). Lean software development—what exactly are we talking about? *Lean Enterprise Software and Systems*, 167, 16–31.
- Choudhury, V., & Sabherwal, R. (2003). Portfolios of control in outsourced software development projects. *Information Systems Research*, 14(3), 291–314.
- Chua, C., Lim, W.-K., Soh, C., & Sia, S. K. (2012). Enacting clan control in complex IT projects: A social capital perspective. *MIS Quarterly*, 36(2), 577–600.
- Cocco, L., Mannaro, K., Concas, G., & Marchesi, M. (2011). Simulating Kanban and Scrum vs. Waterfall with system dynamics. In A. Sillitti, O. Hazzan, E. Bache, & X. Albaladejo (Eds.), *International conference on agile software development* (pp. 117–131). Berlin: Springer.
- Conboy, K. (2009). Agility from first principles: Reconstructing the concept of agility in information systems development. *Information Systems Research*, 20(3), 329–354.

- Conboy, K. (2010). Project failure en masse: A study of loose budgetary control in ISD projects. *European Journal of Information Systems*, 19, 273–287.
- Concas, G., Lunesu, M. I., Marchesi, M., & Zhang, H. (2013). Simulation of software maintenance process, with and without a work-in-process limit. *Journal of Software: Evolution and Process*, 25(12), 1225–1248.
- Csikszentmihalyi, M., & Csikszentmihalyi, I. (1975). *Beyond boredom and anxiety* (Vol. 721). San Francisco, CA: Jossey-Bass.
- Csikszentmihalyi, M., & Kleiber, D. A. (1991). Leisure and self-actualization. In O. L. Driver, P. J. Brown, & G. L. Peterson (Eds.), *Benefits of leisure* (pp. 99–102). State College, PA: Venture Publishing.
- Dennehy, D., & Conboy, K. (2017). Going with the flow: An activity theory analysis of flow techniques in software development. *Journal of Systems and Software*, 133(2017), 160–173.
- Denzin, N. K., & Lincoln, Y. S. (Eds.). (2011). *The Sage handbook of qualitative research*. London, England: Sage Publications.
- Drummond, H. (2011). MIS and illusions of control: An analysis of the risks of risk management. *Journal of Information Technology*, 26(4), 259–267.
- Dybå, T., & Dingsøyr, T. (2008). Empirical studies of agile software development: A systematic review. *Information and Software Technology*, 50(9–10), 833–859.
- Ebert, C., Abrahamsson, P., & Oza, N. (2012). Lean software development. *IEEE Software*, 29(5), 22–25.
- Eisenhardt, K. M. (1989). Building theories from case study research. *The Academy of Management Review*, 14(4), 532–550.
- Fichman, R. G., & Kemerer, C. F. (1997). The assimilation of software process innovations: An organizational learning perspective. *Management Science*, 43(10), 1345–1363.
- Fitzgerald, B., Russo, N. L., & Stolterman, E. (2002). *Information systems development: Methods in action*. New York, NY: McGraw-Hill.
- Fitzgerald, B., & Stol, K.-J. (2015). Continuous software engineering: A roadmap and agenda. *Journal of Systems and Software*, 123(2017), 176–189.
- Gallivan, M. J. (2001). Organizational adoption and assimilation of complex technological innovations: Development and application of a new framework. *ACM SIGMIS Database*, 32(3), 51–85.
- Glaser, B. G., & Strauss, A. (1967). *The discovery of ground theory*. New York, NY: Aldine de Gruyter.
- Harris, M. L., Collins, R. W., & Hevner, A. R. (2009). Control of flexible software development under uncertainty. *Information Systems Research*, 20(3), 400–419.
- Hinchey, M., Fitzgerald, B., Donnellan, B., & Margaria, T. (2016). *Innovation potential of software technologies in the context of Horizon 2020*. Retrieved from <http://www.lero.ie/sites/default/files/DGConnect.pdf>
- Ikonen, M. (2010). Leadership in Kanban software development projects: A quasi-controlled experiment. In P. Abrahamsson & N. Oza (Eds.), *Lean enterprise software and systems* (pp. 85–98). Berlin: Springer.
- Ikonen, M., Kettunen, P., Oza, N., & Abrahamsson, P. (2010). Exploring the sources of waste in Kanban software development projects. In *36th EUROMICRO conference on Software Engineering and Advanced Applications (SEAA 2010)* (pp. 376–381). Los Alamitos, CA: IEEE Computer Society.
- Ikonen, M., Pirinen, E., Fagerholm, F., Kettunen, P., & Abrahamsson, P. (2011). *On the impact of Kanban on software project work: An empirical case study investigation* (pp. 305–314). In IEEE International Conference on Engineering of Complex Computer Systems (ICECCS), Las Vegas, Nevada. doi:10.1109/ICECCS.2011.37
- Khurum, M., Petersen, K., & Gorschek, T. (2014). Extending value stream mapping through waste definition beyond customer perspective. *Journal of Software: Evolution and Process*, 26(12), 1074–1105.
- Kirsch, L. J., Sambamurthy, V., Ko, D. G., & Purvis, R. L. (2002). Controlling information systems development projects: The view from the client. *Management Science*, 48(4), 484–498.
- Kirsch, L. J. (1997). Portfolios of control modes and IS project management. *Information Systems Research*, 8(3), 215–239.
- Kitchenham, B., Pretorius, R., Budgen, D., Brereton, O. P., Turner, M., Niazi, M., & Linkman, S. (2010). Systematic literature reviews in software engineering—A tertiary study. *Information and Software Technology*, 52(8), 792–805.
- Kitchenham, B. A., Pfleeger, S. L., Pickard, L. M., Jones, P. W., Hoaglin, D. C., El Emam, K., & Rosenberg, J. (2002). Preliminary guidelines for empirical research in software engineering. *IEEE Transactions on Software Engineering*, 28(8), 721–734.
- Klipp, P. (2011). *Getting started with Kanban*. Kanbanery. Retrieved from <https://morebetterlabs.kanbanery.com/ebook/GettingStartedWithKanban.pdf>
- Kniberg, H., & Skarin, M. (2010). *Kanban and Scrum-making the most of both*. Retrieved from <https://www.infoq.com/minibooks/kanban-scrum-minibook>
- Law, E. L. C., & Lárusdóttir, M. K. (2015). Whose experience do we care about? Analysis of the fitness of scrum and Kanban to user experience. *International Journal of Human-Computer Interaction*, 31(9), 584–602.
- Leopold, K., & Kaltenecker, S. (2015). *Kanban change leadership*. Hoboken, NJ: John Wiley & Sons. Retrieved from <https://www.wiley.com/en-us/Kanban+Change+Leadership%3A+A+Creating+a+Culture+of+Continuous+Improvement-p-9781119019701>
- Lyytinen, K., & Rose, G. M. (2006). Information system development agility as organizational learning. *European Journal of Information Systems*, 15(2), 183–199.
- Maxwell, J. (1992). Understanding and validity in qualitative research. *Harvard Educational Review*, 62(3), 279–301.
- Mahnic, V. (2015). From Scrum to Kanban: Introducing lean principles to a software engineering capstone course. *International Journal of Engineering Education*, 31(4), 1106–1116.
- Middleton, P., & Joyce, D. (2012). Lean software management: BBC worldwide case study. *IEEE Transactions on Engineering*

- Management*, 59(1), 20–32. doi:10.1109/TEM.2010.2081675; Retrieved from <https://pure.qub.ac.uk/ws/files/724967/Lean%20Software%20Management%20BBC%20Worldwide%20Case%20Study%20Feb%202011.pdf>
- Miles, M. B., & Huberman, A. M. (1984). *Qualitative data analysis*. Beverly Hills, CA: Sage Publications.
- Mujtaba, S., Feldt, R., & Petersen, K. (2010). *Waste and lead time reduction in a software product customization process with value stream maps*. 21st Australian Software Engineering Conference (ASWEC), IEEE Computer Society. Retrieved from <http://www.diva-portal.org/smash/record.jsf?pid=diva2%3A835552&dswid=2813>
- Naylor, J. B., Naim, M. M., & Berry, D. (1999). Leagility: Integrating the lean and agile manufacturing paradigms in the total supply chain. *International Journal of Production Economics*, 62(1–2), 107–118.
- Nikitina, N., & Kajko-Mattsson, M. (2011). *Developer-driven big-bang process transition from Scrum to Kanban* (pp. 159–168). Waikiki, Honolulu, HI: Proceedings of the 2011 International Conference on Software and Systems Process.
- Nord, R. L., Ozkaya, I., & Sangwan, R. S. (2012). Making architecture visible to improve flow management in lean software development. *IEEE Software*, 29(5), 33–39.
- Ohno, T. (1988). *Toyota production system: Beyond large-scale production*. Boca Raton, FL: CRC Press.
- Olerup, A. (1991). Design approaches: A comparative study of information system design and architectural design. *The Computer Journal*, 34(3), 215–224.
- Oppenheim, A. (1992). Questionnaire design, interviewing and attitude measurement. *Pinter, London*, 4(5), 371–382.
- Orlikowski, W. J. (1993). CASE tools as organizational change: Investigating incremental and radical changes in systems development. *MIS Quarterly*, 17(3), 309–340.
- Petersen, K. (2011). Is lean agile and agile lean? A comparison between two software development paradigms. *Modern Software Engineering Concepts and Practices: Advanced Approaches*, 1, 19–46. doi:10.4018/978-1-60960-215-4.ch002
- Petersen, K., Roos, P., Nyström, S., & Runeson, P. (2014). Early identification of bottlenecks in very large scale system of systems software development. *Journal of Software: Evolution and Process*, 26(12), 1150–1171.
- Petersen, K., & Wohlin, C. (2009). *Context in industrial software engineering research* (pp. 401–404). *Proceedings of the 2009 3rd International Symposium on Empirical Software Engineering and Measurement*, IEEE Computer Society, Washington, DC.
- Petersen, K., & Wohlin, C. (2011). Measuring the flow in lean software development. *Software: Practice and Experience*, 41(9), 975–996.
- Ply, J. K., Moore, J. E., Williams, C. K., & Thatcher, J. B. (2012). Is employee attitudes and perceptions at varying levels of software process maturity. *MIS Quarterly*, 36(2), 601–624.
- Polk, R. (2011). *Agile and Kanban in coordination* (pp. 263–268). Proceedings of the Agile Conference, Salt Lake City, UT. doi:10.1109/AGILE.2011.10
- Poppendieck, M. (2011). Principles of lean thinking. *IT Management Select*, 18(1), 1–7. Retrieved from <http://www.sel.unsl.edu.ar/ApuntesMaes/Anteriores/MetodologiasAgiles/LeanThinking.pdf>
- Poppendieck, M., & Cusumano, M. A. (2012). Lean software development: A tutorial. *IEEE Software*, 29(5), 26–32.
- Poppendieck, M., & Poppendieck, T. (2013). *The lean mindset: Ask the right questions*. London, England: Pearson Education.
- Power, K. (2014). Definition of ready: An experience report from teams at Cisco. In G. Cantone & M. Marchesi (Eds.), *Agile processes in software engineering and extreme programming. XP 2014. Lecture notes in business information processing* (Vol. 179, pp. 234–241). Rome, Italy: Springer Science + Business Media. doi:10.1007/978-3-319-06862-6_25
- Power, K., & Conboy, K. (2015). A metric-based approach to managing architecture-related impediments in product development flow: An industry case study from Cisco. In Bertolino Antonia (Ed.), *Proceedings of the second international workshop on software architecture and metrics* (pp. 15–21). Florence, Italy: IEEE Press.
- Reinertsen, D. G. (2009). *The principles of product development flow: Second generation lean product development*. Redondo Beach, CA: Celeritas.
- Remenyi, D., & Williams, B. (1995). Some aspects of methodology for research in information systems. *Journal of Information Technology*, 10(3), 191–201.
- Robinson, H., Segal, J., & Sharp, H. (2007). Ethnographically-informed empirical studies of software practice. *Information and Software Technology*, 49(6), 540–551.
- Rubin, H., & Rubin, I. (2005). *Qualitative interviewing: The art of hearing data*. Thousand Oaks, CA: Sage Publications.
- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164.
- Rutherford, K., Shannon, P., Judson, C., & Kidd, N. (2010). From chaos to Kanban, via scrum (pp. 344–352). In A. Sillitti, A. Martin, X. Wang, & E. Whitworth (Eds.), *International conference on agile software development*. Berlin: Springer.
- Senapathi, M., Middleton, P., & Evans, G. (2011). Factors affecting effectiveness of agile usage—Insights from the BBC worldwide case study (pp. 132–145). In A. Sillitti, O. Hazzan, E. Bache, & X. Albaladejo (Eds.), *International conference on agile software development*. Berlin: Springer.
- Shinkle, C. M. (2009). *Applying the Dreyfus model of skill acquisition to the adoption of Kanban systems at Software Engineering Professionals (SEP)* (pp. 186–191). Agile Conference, AGILE'09, Chicago, IL. doi:10.1109/AGILE.2009.25
- Sjöberg, D. I., Johnsen, A., & Solberg, J. (2012). Quantifying the effect of using Kanban versus scrum: A case study. *IEEE Software*, 29(5), 47–53.
- Stake, R. (2000). *Case studies in handbook of qualitative research*. Thousand Oaks, CA: Sage Publications.
- Standish Group. (2015). *Standish CHAOS report*. Retrieved from <https://www.projectsmart.co.uk/white-papers/chaos-report.pdf>. Accessed September 6 2018.
- Stray, V. G., Moe, N. B., & Dingsøyr, T. (2011). Challenges to teamwork: A multiple case study of two agile teams. In: A. Sillitti, O.

- Hazzan, E. Bache, & X. Albaladejo (Eds.), *International conference on agile software development* (pp. 146–161). Berlin: Springer.
- Towill, D., & Christopher, M. (2002). The supply chain strategy conundrum: To be lean or agile or to be lean and agile? *International Journal of Logistics*, 5(3), 299–309.
- VersionOne. (2016). *What is Kanban?* Retrieved from <https://www.versionone.com/what-is-kanban/>
- Wastell, D., & Newman, M. (1993). The behavioral dynamics of information system development: A stress perspective. *Accounting, Management and Information Technologies*, 3(2), 121–148.
- Wengraf, T. (2001). *Qualitative research interviewing: Biographic narrative and semi-structured methods*. Thousand Oaks, CA: Sage Publications.
- Wiener, M., Remus, U., Heumann, J., & Mähring, M. (2015). The effective promotion of informal control in information systems offshoring projects. *European Journal of Information Systems*, 24(6), 569–587.
- Womack, J. P., & Jones, D. T. (2010). *Lean thinking: Banish waste and create wealth in your corporation*. New York, NY: Simon & Schuster.
- Womack, J. P., Jones, D. T., & Roos, D. (1990). How lean production can change the world. *New York Times Magazine*, 13(23), 20–38.
- Yin, R. K. (2009). *Case study research: Design and methods (applied social research methods)*. London, England and Singapore: Sage Publications.

Author Biographies

Denis Dennehy is a research fellow with Lero | The Irish Software Research Centre at National University of Ireland Galway (NUIG), Galway, Ireland. He has a keen interest in current discourse on the sociotechnical aspects of information systems development (ISD) and the emphasis on viewing ISD as an evolving activity system beyond a single user. He has published work in leading international journals and conferences, including *Information & Management*, *Journal of Systems & Software*, and the *International Conference on Information Systems*. He is a senior editor of information technology and people and program director of the MSc business analytics program at NUIG. He can be contacted at denis.dennehy@nuigalway.ie

Kieran Conboy is a professor with Lero | The Irish Software Research Centre at National University of Ireland Galway, Galway, Ireland. Kieran has published more than 150 articles in leading international journals and conferences, including *Information Systems Research*, the *European Journal of Information Systems*, and the *Journal of the AIS*. His research examines contemporary technology management and design, including concepts such as temporality, flow, open innovation, and agility. He is an editor of the *European Journal of Information Systems* and has chaired many international conferences in his field. He can be contacted at kieran.conboy@nuigalway.ie